

Software engineering

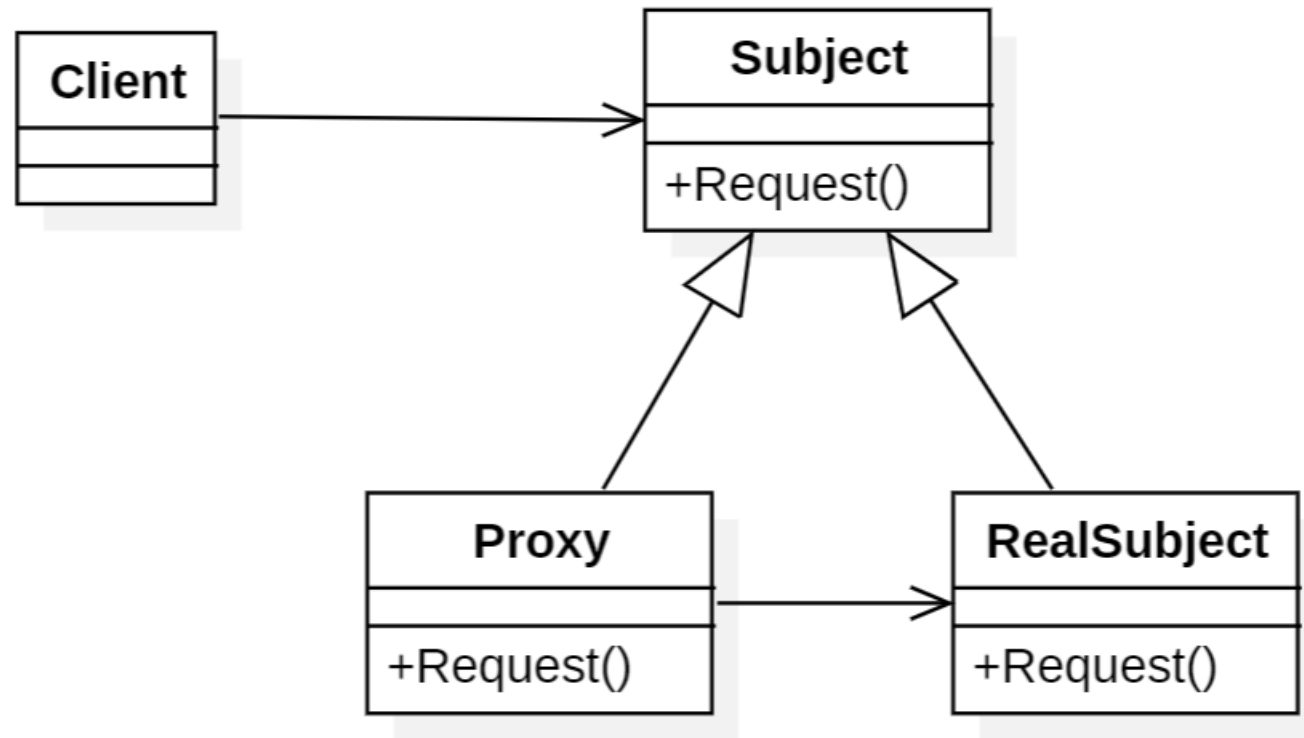
Moving to a server with ORM

Skill progressions by applying material:

Database access +1

Web development +0-1

Proxy



```
public static void Main(string[] args)
{
    Proxy proxy = new Proxy();
    proxy.Request();
}

public abstract class Subject
{
    public abstract void Request();
}

public class RealSubject : Subject
{
    public override void Request()
    {
        Console.WriteLine("Called RealSubject.Request()");
    }
}

public class Proxy : Subject
{
    private RealSubject realSubject;
    public override void Request()
    {
        if (realSubject == null)
        {
            realSubject = new RealSubject();
        }
        realSubject.Request();
    }
}
```

Entity framework and API server



ASP.NET web API steps

- Add ASP.NET Core Web API project
- Manage nuget packages; add Swagger, SwaggerGen, and SwaggerUI
- Right click your solution, select multiple start up projects
- In your program.cs add:
 - `builder.Services.AddSwaggerGen();`
 - `app.UseSwagger();`
 - `app.UseSwaggerUI();`
- Write your controllers (example next slide)
- Run then go to [https://localhost:\[portNumber\]/swagger/index.html](https://localhost:[portNumber]/swagger/index.html)

API controller example

```
[ApiController]
[Route("api/[controller]")]
public class GameTradeController : ControllerBase
{
    IRepository repository = new Repository();

    [HttpGet(Name = "GetGameTrade")]
    public IEnumerable<GameTrade> GetGameTrades()
    {
        return repository.GetGameTrades().ToList();
    }
}
```

Class library [optional but recommended]

- Add class library project
- Move your Repositories to it
- Move your IRepository to it
- Right click Dependencies on your WinUI, Web API, and Testing projects and add project reference then select the new project
- Set up entity framework here

Entity framework

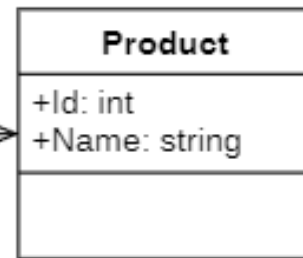
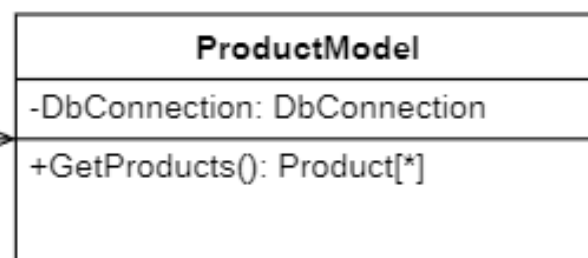
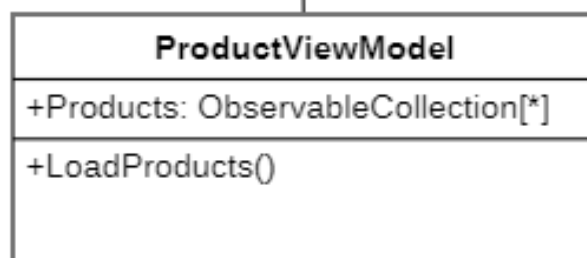
- Add EntityFrameworkCore + EntityFrameworkCoreSqlServer nuget packages
- Add a DbContext (example next slide)
- In appsettings.json add: "ConnectionStrings": { "connection string identifier": "your connection string here"},
- In program.cs add:
`builder.Services.AddDbContext<YourDbContextType>(options => options.UseSqlServer(builder.Configuration.GetConnectionString("connection string identifier")));`
- In package manager console type
 - add-migration [current date as string] number
 - update-database

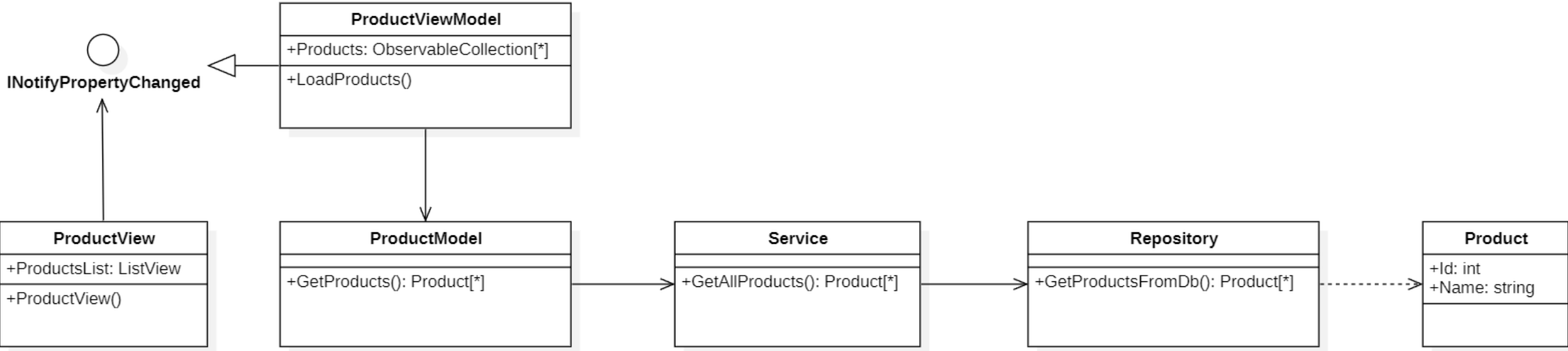
Db context example

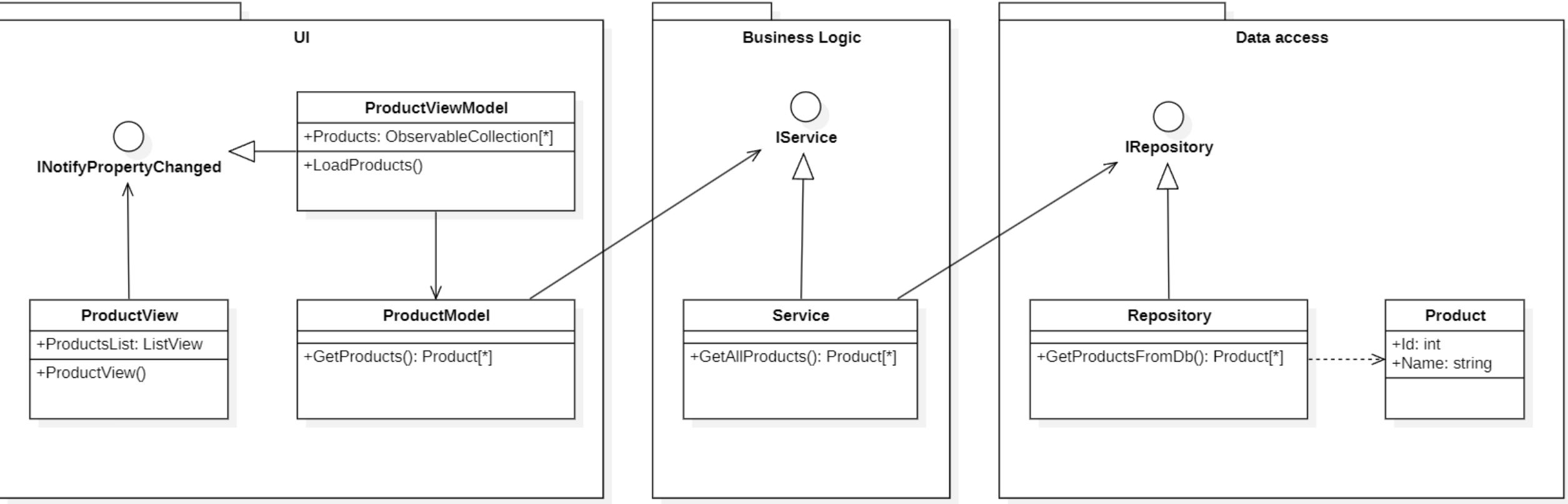
```
public class [projectName]Context : DbContext
{
    public [projectName]Context(DbContextOptions<[projectName]Context> options)
        : base(options)
    {
    }

    public DbSet<Entity> Entities { get; set; } = default!;
}
```

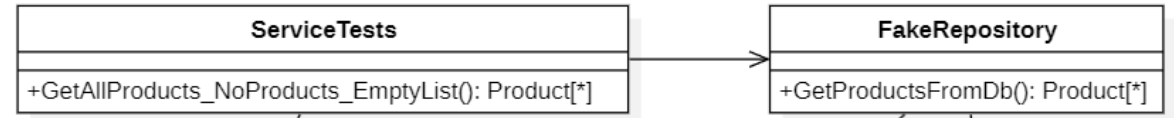
INotifyPropertyChanged



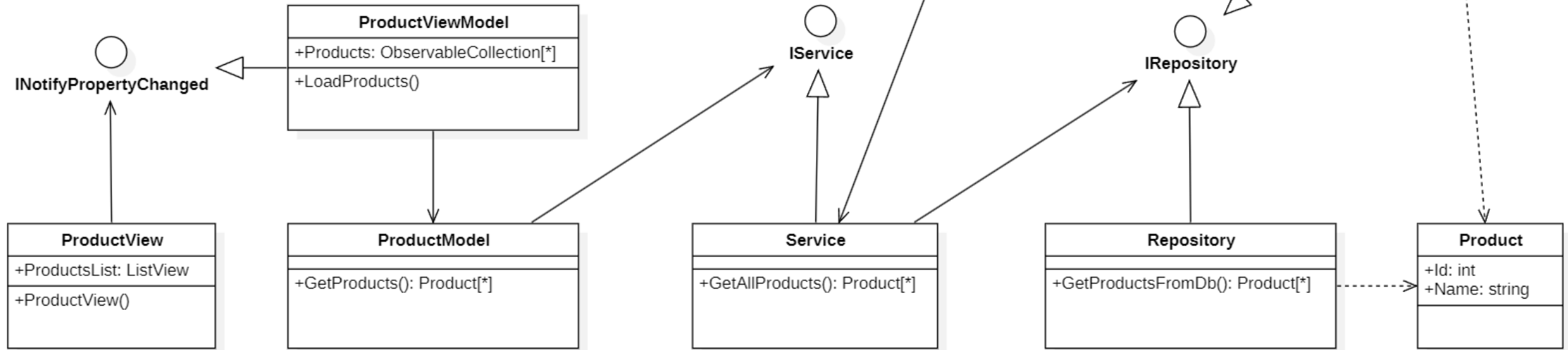




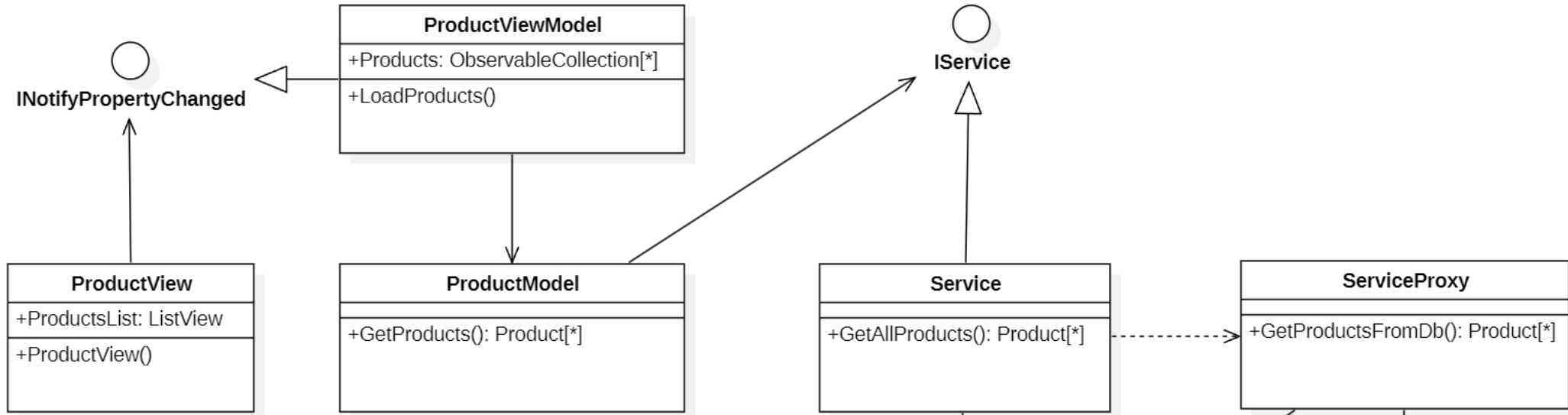
Testing project



Application project



Application project



Server API project

